Unit III

3 Linkers and Loaders

3.1 : Introduction

Q.1 What is loader ?

Ans. : Loader is a utility program which takes object code as input prepares it for execution and loads the executable code into the memory. Thus loader is actually responsible for initiating the execution process.

Q.2 Discuss four different functions of loader.

☞ [SPPU : April-18, Marks 4]

Ans. : The loader is responsible for the **activities** such as allocation, linking, relocation and loading.

1) It allocates the space for program in the memory, by calculating the size of the program. This activity is called **allocation**.
2) It resolves the symbolic references (code/data) between the object modules by assigning all the user subroutine and library subroutine addresses. This activity is called **linking**.
3) There are some address dependent locations in the program, such address constants must be adjusted according to allocated space, such activity done by loader is called **relocation**.
4) Finally it places all the machine instructions and data of corresponding programs and subroutines into the memory. Thus program now becomes ready for execution, this activity is called **loading**.

3.2 : Loader Schemes

Q.3 What are types of loaders ? Explain compile and go loader scheme with advantages and disadvantages. ☞ [SPPU : April-18, Marks 6]

Ans. : Types of Loaders :

Based on various functionalities of the loader, there are various types of loaders, which are known as loading schemes. Various loaders schemes are -

1. Compile and Go
2. General Loader Scheme
3. Absolute Loader
4. Subroutine Linkage
5. Relocating Loaders
6. Direct Linkage Loader

Compile and go Loader :

- In this type of loader, the instruction is read line by line, its machine code is obtained and it is **directly put in the main memory** at some **known address.**
- That means the assembler runs in one part of memory and the assembled machine instructions and data is directly put into their assigned memory locations.
- After completion of assembly process loader contains the instruction using which the location counter is set to the start of the newly assembled program.
- The typical example is WATFOR-77, its a FORTRAN compiler which uses such "load and go" scheme. This loading scheme is also called as **"assemble and go".**

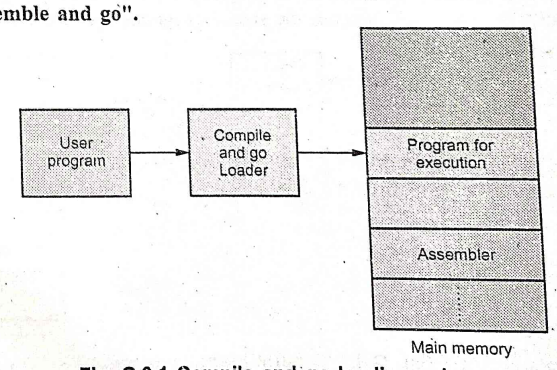

Fig. Q.3.1 Compile and go loading scheme

Advantage : It is simple to implement.

Disadvantages : 1) Assembler occupies memory. Hence memory space remains occupied.

2) There is no production of .obj file, the source code is directly converted to executable form.

Q.4 Explain general loading scheme (using suitable diagram) with advantages and disadvantages.

☞ [SPPU : Dec.-18, End Sem, March-20, In Sem, Marks 5, April-19, In Sem, Marks 6]

Ans. : In this loader scheme, the source program is **converted to object program** by some translator (assembler).

- The loader accepts these object modules and puts machine instruction and data in an executable form at their assigned memory.
- The loader occupies some portion of main memory.

Advantages

1. The program need not be retranslated each time while running it. This is because initially when source program gets executed an object program gets generated. If program is not modified, then loader can make use of this object program to convert it to executable form.
2. There is no wastage of memory, because assembler is not placed in the memory, instead of it, loader occupies some portion of the memory. And size of loader is smaller than assembler, so more memory is available to the user.
3. It is possible to write source program with multiple programs and multiple languages, because the source programs are first converted to

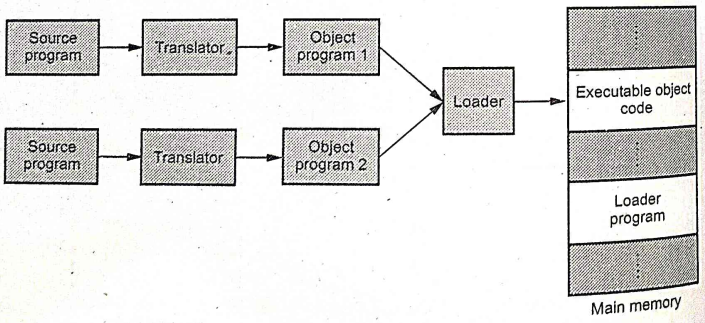

Fig. Q.4.1 General loader scheme

object programs always, and loader accepts these object modules to convert it to executable form.

Q.5 What are subroutine linkages ? What are the benefits of using it ? ☞ [SPPU : May-19, Marks 4]

Ans. : Subroutine Linkage : It is a loading scheme which makes it possible to call and return from subroutines.

In this mechanism the assembler pseudo-op **EXTRN** is followed by list of symbols. These are basically the symbols which are defined externally but referenced in the present program.

Similarly if a symbol is defined in one program and is referenced in others, then this type of symbol is preceded by pseudo-op **ENTRY** in the symbol table.

For example -

```
MAIN    START
        EXTRN   B  //subroutine B is defined externally
        -----
        -----
        L               15,A(B)
        BALR            14,15
        .
        .
        .
        END
```

```
B       START
        USING           *,15
        .
        .
        BR              14
        END
```

Assembler Linkage Pseudo-Ops

Suppose we have a main program named **A** and it uses variables A1,A2,A3.. which are defined in the same program then the pseudo-op ENTRY is used for their declaration

```
A       START
        ENTRY       A1,A2,A3

A1      -----
A2      -----
```

Subroutine Reference(Extrn Pseudo-Op)

Assembler symbols can be internal or external. External symbols are those symbols whose value is not known to the assembler but will be provided by loader. These symbols are referred by the assembler using **EXTRN**

For example -

EXTRN B1,B2

Here B1 and B2 are the extern symbols.

Q.6 Explain : Absolute loader. ☞ [SPPu : Dec.-19, Marks 2]

Ans. : • Absolute loader is a kind of loader in which relocated object files are created, loader accepts these files and places them at specified locations in the memory.

• This type of loader is called **absolute** because no relocation information is needed; rather it is obtained from the programmer or assembler.

Q.7 What is the difference between Absolute loader and Compile and Go loader ?

Ans. : Difference between Absolute and Compile and Go loader :

Absolute loader	Compile and Go loader
The assembled code is placed in relocated object files. Then loader accepts these files and object code is place at some assigned memory location.	The assembled code is directly summed into the memory at some assigned memory location.
This scheme allows to load object modules of multiple programs written in different languages.	This scheme does not allow object modules of different source programs.
This is efficient scheme.	As in this type of loader the instruction is read line by line, its object code is obtained and it is then directly put into the memory, this scheme is less efficient but simple to implement.

The drawback of this method is that assembler and loader occupies large block of memory. Hence there is a wastage of memory.	In this scheme it is programmers duty to adjust all the inter segment addresses and manually do the linking activity. If at all any modification is done in some segment then starting address of next subsequent segment changes. The programmer has to take care of all these issues.

Q.8 What are the advantages and disadvantages of relocating loader ?

Ans. : Advantages :

1. Using branch instruction to corresponding subroutine the desired subroutine can be accessed. Thus the four functions of loader i.e. allocation, linking, relocation and loading are performed **automatically** by the loader.
2. Using **relocation bits** it can be identified that which instruction needs to be relocated and which instruction needs to be directly placed.
3. Using transfecter vector linking of external subroutines can be done with the main procedure. Thus transfer vector of relocating loader helps in solving the external references.
4. In this loader scheme, the assembler provides the additional information such as length of entire program and length of transfer vector to the loader. Using this information allocation problem gets solved.

Disadvantages :

1) The transfer vector links are useful for transferring the control to external subroutines but this vector table is not well-suited for loading or storing external data (i.e. data present in another program segment).
2) Due to transfer vector, the size of object program in the memory gets increased.
3) The relocating loader handles the shared procedure segments but it can not handle the shared data segments.

Q.9 Explain working of Binary Symbolic Subroutine(BSS) loader with example. [SPPU : Dec.-19, Marks 6]

Ans. : • When a single subroutine is changed then all the subroutines need to be reassembled. The task of allocation and linking must be carried out once again. To avoid this rework a new class of loaders is introduced which is called **relocating loader.**

- **Example :** The **Binary Symbolic Subroutine** (BSS) loader used in IBM 7094 machine is a relocating loader.
- In BSS loader there are **many procedure segments** and **one common data segment.**
- The assembler reads the source program, assembles each procedure segment independently. This information along with intersegment references is passed on to loader.
- The output of relocating loader is the object program and the program references. In addition to this the **relocation information** is given. Relocation information is the information about the locations that need to be changed. If it is to be loaded in an arbitrary place in core.

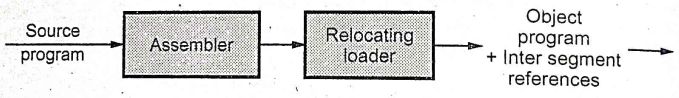

Fig. Q.9.1 Relocating loader

- The assembler takes the source program as input, this source program may call some external routines. Hence assembler produces the object code as an output along with a table. This table contains entries of the external references made in the program. This table is called **transfer vector.**

For example : If a program refers a routine for summing up the numbers (SUM) then transfer vector would contain the symbolic name **SUM.**

The statement calling **SUM** would be translated into a transfer instruction indicating the pointer to the location of the transfer vector associated with **SUM.**

- The assembler also provide some additional information to the loader which includes :
  - Length of entire program.
  - Length of transfer vector portion.
- The loader loads the object program and transfer vector into core. Then it would load each subroutine identified in the transfer vector.
- The loader then place the transfer instruction to the corresponding subroutine for each entry in transfer vector. Thus execution of the **Call for SUM** routine will result in branch to the first location in transfer vector. This location will provide transfer instruction to the location of **SUM.**
- The BSS loader scheme is normally used on the computers with a fixed length direct address instruction format.

Q.10 Explain the concept of transfer vector in BSS loader

Ans. : • The assembler takes the source program as input, this source program may call some external routines. Hence assembler produces the object code as an output along with a table. This table contains entries of the external references made in the program. This table is called **transfer vector.**

For example : If a program refers a routine for summing up the numbers (SUM) then transfer vector would contain the symbolic name **SUM.**

Q.11 Give the direct address instruction format for BSS loader scheme

Ans. : • The BSS loader scheme is normally used on the computers with a fixed length direct address instruction format. This format is as follows -

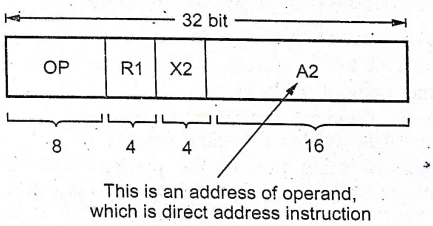

Fig. Q.11.1

Where A2 is 16 bit absolute address of operand. This is a direct address instruction format. This format is useful if there are less than $2^{16} = 65,356$ bytes of storage.

Q.12 Define the term - Relocation bit.

Ans. : Some instructions in the program source need to be relocated while other need not be relocated. Assembler associates a bit called **relocation bit** with each instruction or address. If this bit is equal to 1 then that means corresponding address field must be relocated, otherwise that address field need not be relocated.

Q.13 Which information does the assembler submit to the direct linking loader ?

Ans. : The source program is read by assembler and **assembler submits following information to loader -**

i) The length of program/segment.

ii) A list of symbols in the segment which may be referenced by other segment and relative address of these symbols within the segment.

iii) A list of all the symbols not defined in the segment but referred by other segments.

iv) The information about the locations of address constants, in the segment along with the description of how to revise them.

v) The translated machine code along with the assigned relative address.

3.3 : Overlay Structure

Q.14 Explain overlay structure. ☞ [SPPU : Dec.-19, Marks 2]

Ans. : Refer Q.15.

Q.15 Write short note on - Overlay structure

Ans. :

- **Definition of Overlay :** Overlay is a part of the program that is currently required for the execution of program and it has the same load origin as other parts of the program.
- **Definition of Overlay Structure :** The program containing multiple overlays is called overlay structured program.
- There might be some part of the program that need to be loaded permanently in the memory for execution of other parts of the program. Such a permanent resident part of the program is called **root.** Other parts of the program can be loaded in the memory as per the requirements.

- In assembler, the overlay structure is used in assembler.
- Following Fig. Q.15.1 represents the concept -

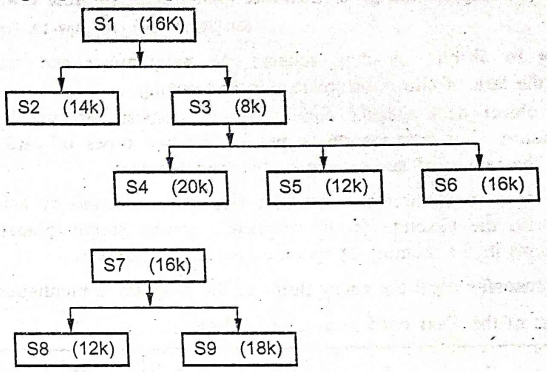

Fig. Q.15.1 Overlay structure

- It is clear from the figure that some parts of the program can reside in the memory simultaneously. Thus the total memory allocated is 78K with overlay structure whereas the memory requirement of entire program is 132K. Thus lot of space can be saved using overlay structuring.

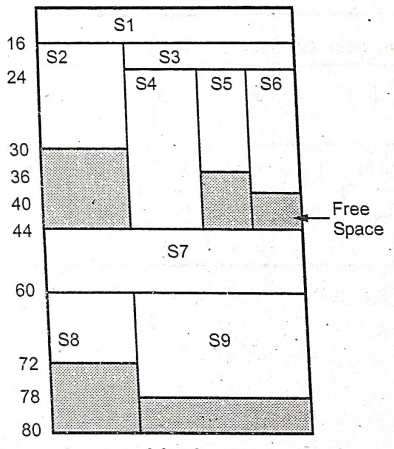

Fig. Q.15.2 Segments stored in the memory using overlay structure

3.4 : Design of Absolute Loader

Q.16 Give complete design of absolute loader with suitable example.

☞ [SPPU : Dec.-18, May-19, Marks 6]

Ans. : • In absolute loading scheme the programmer and assembler perform the task of allocation, relocation and linking.

• Every object deck (object code program) consists of two types of information. This information is present on two types of cards. First type is the **text card** and second one is **transfer card**.

• On the **text card**, there are machine instructions created by assembler along with the absolute (core) addresses. Loader simply places these instructions in the memory at specified absolute addresses.

• On the **transfer card** the **entry point** of the program is mentioned.

The format of the **Text card** is as given below

Column	1	2	3 to 5	6 - 7	8 - 72	73 - 80
Contents	Type of card (= 0 for text card)	Count of information (i.e. length of the code)	Addresses at which the translated code must be place.	Empty	Instructions and data to be loaded in memory	Card sequence number

The format of **Transfer card** is as follows -

Column Number	1	2	3 - 5	6 - 72	73 - 80
Contents	Type of card (= 1 for transfer card)	Count = 0	Address of entry point	Empty	Card sequence number

When the card is read it is stored in 80 continuous bytes in memory.

• Design of absolute loader can be illustrated by following flowchart -

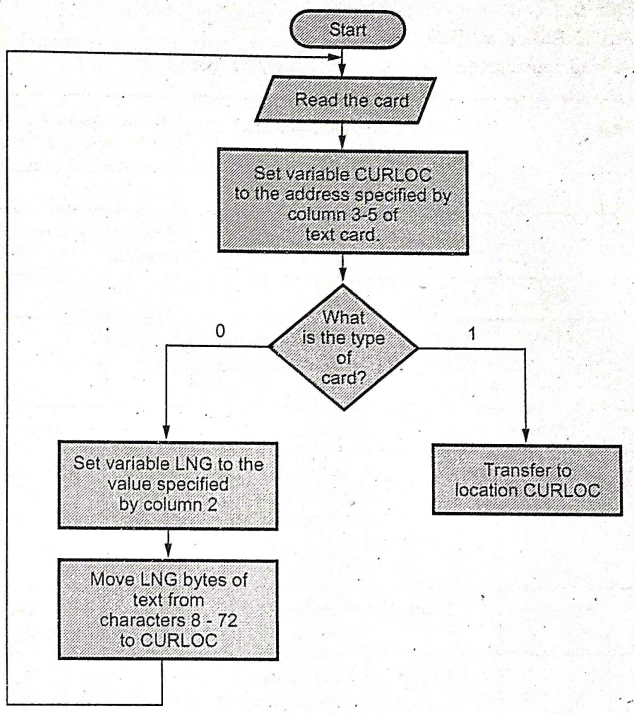

Fig. Q.16.1 Flowchart for design of absolute loader

3.5 : Design of Direct Linking Loader

Q.17 Explain the formats of ESD, RLD, TXT and END cards with respect to direct linking loader with suitable example.

☞ [SPPU : May-19, Marks 6]

Ans. : The four types of cards produced during direct linking loader are -

ESD,TXT,RLD and END

Following source code is considered as input to assembler.

Line No.			Comment	The translated code will be produced by assembler and is as given below -	
				Relative location	Object code
1.	MAIN	START	Start		
2.	ENTRY	RESULT	Locally defined subroutine (entry point)		
3.	EXTRN	SQRT	External subroutine		
4. 5.	BALR USING	12, 0 *, 12	Set register 12 as base address and instruct assembler to use it as base register.	0	BALR 12,0
6.	ST	14, TMPLOC	Store return address	2	ST 14, 54 (0, 12)
7.	L	1, PTRVAL	Load register 1 with the constant PTRVAL i.e. with table values	6	L 1, 46 (0,12)
8.	L	15, ASQRT	Load register 15 with the constant ASQRT	10	L 15, 58 (0, 12)
9.	BALR	14, 15	Call subroutine SQRT	14	BALR 14, 15

10.	ST	1, RESULT	Store return address in RESULT	16	ST 1, 50 (0, 12)
11.	L	14, TMPLOC	Get return address in register 14	20	L 14, 54 (0, 12)
12.	BR	14	Return to caller subroutine.	24	BCR 15, 14
				26	--
13.	TABLE	DC F' 11, 12, 13, 14, 15'	Table is defined by constants 11, 12, 13, 14, 15	28	11
				32	12
				36	13
				40	14
				44	15
14.	PTRVAL	DC A (TABLE)	Address of Table is stored in a constant which is labelled as PTRVAL	48	28
15.	RESULT	DS F		52	--
16.	TMPLOC	DS F		56	--
17.	ASQRT	DC A(SQRT)	Storing address of subroutine SQRT in a constant	60	?
18.		END		64	-

ESD : The External Symbol Dictionary (ESD) card contains information about all external symbols. These symbols are of two types -

i) The symbols are defined in this program but referred elsewhere

ii) The symbols referred in this program but defined elsewhere.

For above given source code, the ESD card will be -

Reference Number	Symbol	Type	Relative location	Length
1.	MAIN	SD	0	64
2.	RESULT	LD	52	-
3.	SORT	ER	-	-

The first entry is for the MAIN program, the type is Segment Definition (SD), the relative location of this program is 0, and total length is 64 bytes. The second entry is for locally defined symbol RESULT. The type is Local Definition (LD). The relative location is 52.

Then the entry for the symbol SQRT is made. This segment is an External Reference (ER). The location and length of this subroutine is not known. Hence kept blank. These entries can be obtained from RLD cards.

- **TXT :** This text card contains the translated code produced by assembler.
  For above discussed example the TXT card will be -

TXT card

Reference Number	Relative Location	Object Code
4	0	BALR 12, 0
6	2	ST 14, 54 (0, 12)
7	6	L 1, 46 (0, 12)
8	10	L 15, 58 (0, 12)
9	14	BALR 14, 15
10	16	ST 1, 50 (0, 12)
11	20	L 14, 54 (0, 12)
12	24	BCR 15, 14
13	26	-
13	28	11
13	32	12

13	36	13
13	40	14
13	44	15
14	48	28
17	60	0

- **RLD** The Relocation and Linkage Directory (RLD) contains following information :
  1) The relative location
  2) By what the constant must be changed to
  3) The operation to be performed.

The RLD Card for above example will be

RLD card

Reference Number	Symbol	Flag	Length	Relative location
14	MAIN	+	4	48
17	SQRT	+	4	60

The + sign denotes that something must be added to the constants. The two constant symbols MAIN and SQRT are relocated. The relative address of MAIN is 48 and that of SQRT is 60.

- **END** : The END card indicates end of object deck/program

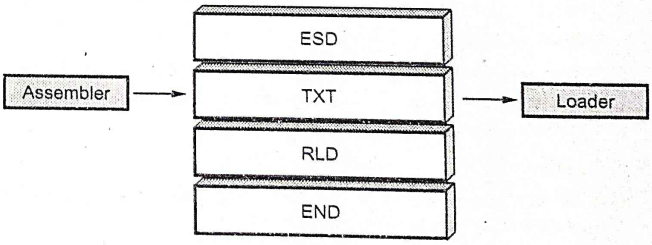

Fig. Q.17.1 Four types of cards

- The assembler submits the information in these four cards to the loader.

Q.18 Explain the working of direct linking loader by Pass1 algorithm and flowchart.

Ans. : Algorithm :

Pass 1 : Symbol definition and allocation of segments

- In this pass the location for each segment is defined. The values to external symbols are assigned. The amount of core storage required for total program must be minimized.

1) The first program segment must be loaded at the address Initial Program Load Address (IPLA). This address is normally specified by operating system.

2) An object card is read and a copy is prepared for pass 2 to read. The object card can be of ESD, TXT, RLD, END or LDT/EOF type.

i) If the card is of **TXT** or **RLD** type then no processing is done. Read the next card.

ii) If the card is of **ESD** type then the processing will be based on SD, LD or ER. If the type of the symbol is **SD** then **length** field is read in variable SLENFGTH. The value is in variable VALUE which is equal to Program Load Address (PLA).

Then it is checked whether this symbol entry is made in **GEST** or not. If symbol entry is not there in GEST then store the symbol name and value. The symbol and value is printed as load map. Similar process is carried out for **LD**. The ER symbols do not require processing.

iii) If the card is of type **END** then the load address of program is incremented by length of the segment. This length is stored in variable.

SLENGTH

iv) If the card is of type LDT or EOF then pass 1 is supposed to be completed and go to pass 2.

Flowchart for pass 1 :

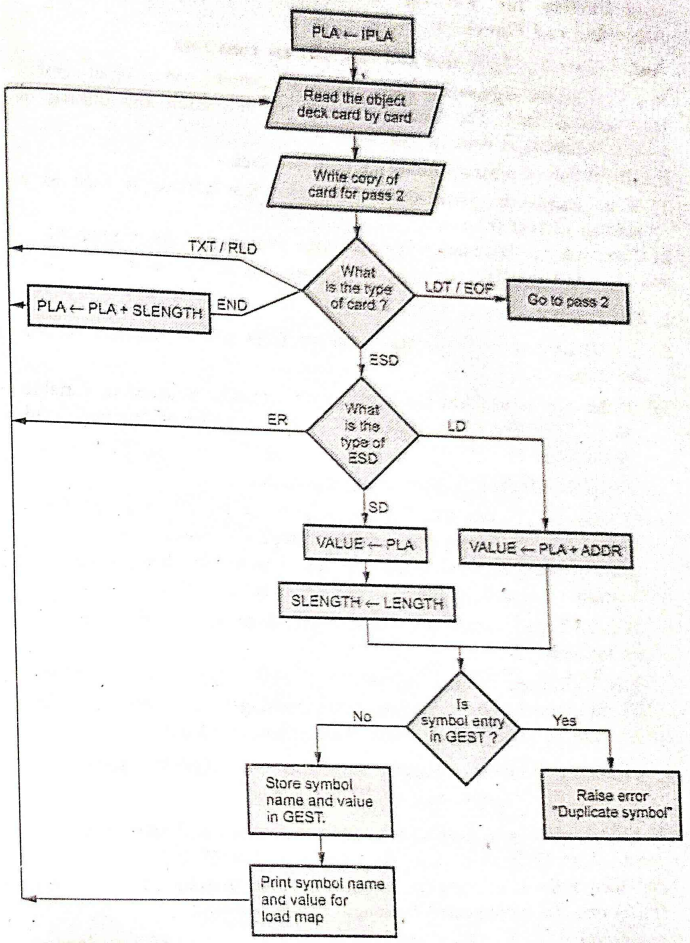

Fig. Q.18.1 Flowchart for Pass 1

Q.19 Explain the working of direct linking loader by Pass2 Algorithm and Flowchart

Ans. : Pass 2 : Load text and link address constants

In pass 1 all the segments get assigned with locations and external symbol table gets defined. The loading of text and relocation and linking of address constants is done in pass 2.

For Execution of program following rules are used -

1) If an address is specified on END card that address is used as a starting address for execution.
2) Otherwise the execution will begin from starting address of segment.

The variable EXADDR will store starting address of execution.

1. The object deck is used card by card.
2. If ESD card is read then based on the ESD type appropriate actions are taken -
   i) If the type is **SD** then the length of the segment is saved in variable SLENGTH. The LESA (ID) is set to current value of Program Load Address.
   ii) If type is **LD**, then no processing is required.
   iii) If type is ER, then GEST entry is searched for ER symbol. If the entry is not found the error message is issued. But if the corresponding external symbol is found in GEST then its value is extracted and corresponding LESA(ID) is set with this value.
3. When TXT card is read the text is copied from TXT card to relocated core location.
4. When RLD Card is read, the relocation and linking values of LESA (ID) are extracted and depending upon the flag values these relocation values can be added or subtracted from address constants

∴ **Actual address const = PLA + ADDR field specified on RLD Card.**

5. If END card is read, then check whether an execution start address is specified on END card; if so, then first add it to PLA (i.e. relocation) and then store it in variable EXADDR. The Program Load Address (PLA) must be incremented by length of the segment.
6. If LDT/EOF card is read, the loader transfers the control to loaded program.

Flowchart for Pass 2 :

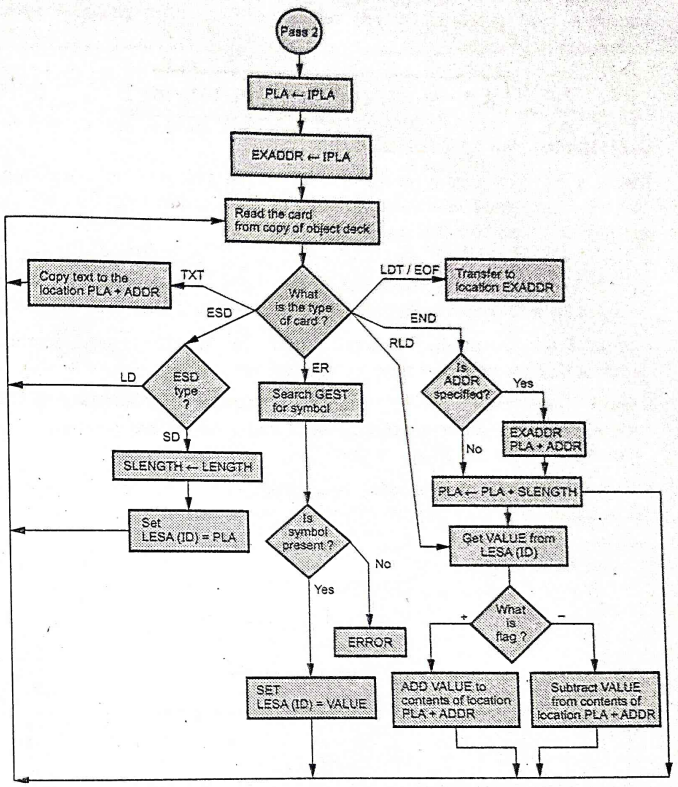

Fig. Q.19.1 Flowchart for Pass 2

3.6 : Self-relocating Programs

Q.20 Explain the term self relocating program.

Ans. : The **self relocatable program** is a program which itself performs relocation. It does not require linkage editor to perform relocation. The self relocating program cannot load itself but relocates itself. The self

relocating program does not bound to a specific memory area for its execution. For preparing a self relocating form of a program special techniques are needed.

3.7 : Static and Dynamic Linking

Q.21 Explain the need for DLL.

Ans. : • DLLs provide a mechanism for shared code and data, allowing a developer of shared code/data to upgrade functionality without requiring applications to be re-linked or re-compiled.

- From the application development point of view Windows can be thought of as a collection of DLLs that are upgraded, allowing applications for one version of the OS to work in a later one.
- kernel32.dll, the primary dynamic library for Windows' base functions such as file creation and memory management
- Many font resource files are also dynamic link libraries having extension .FON. They contain no code and data but have various fonts that any window program can use.
- Thus purpose of dynamic link library is to provide various functionalities and resources required by many different programs.

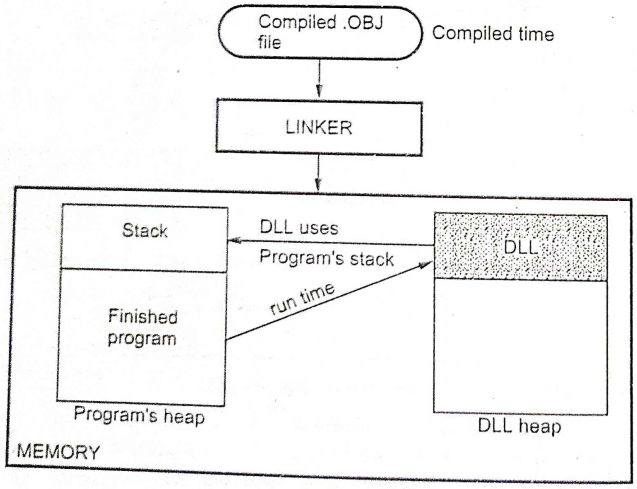

Fig. Q.21.1 Use of DLL

Q.22 What is the difference between DLL and EXE ?

Ans. :

Sr. No.	.DLL	.EXE
1.	It contains code and data.	It contains an executable code.
2.	The .DLL file is linked or referenced to the .EXE at run time.	.EXE runs on its own.
3.	The .DLL extension stands for dynamic link library file.	The .EXE extension stands for an executable file.
4.	It is a library file that can be used by many programs at the same time.	It is an executable file.

Q.23 Differentiate between static and dynamic link libraries.

☞ [May-19, March-20, Marks 4]

Ans. :

Sr. No.	Static link libraries	Static link libraries
1.	For static library, actual code is extracted from library by linker and final executable code is built at the compilation time.	For dynamic link library code need not be extracted and copied rather it is linked with executable code at run time.
2.	Compatibility issues do not arise.	Compatibility issues may arise.
3.	Size is larger.	It is compact in size.
4.	Examples - All the .lib files.	Examples - All the .dll files.

Q.24 What is the need of DLL ? Differentiate between static and dynamic linking.

☞ [SPPU : April-18, Marks 4]

Ans. : Refer Q.21 and Q.23.

END...